

Target Industry and Business Problem

- **Industry:** Quick-Commerce, Urban Warehousing and Last-Mile Electric Fleet Logistics
- **Problem Statement:** Real-time predictive inventory balancing and green-fleet energy management for micro-fulfillment hubs handling hyper-local 10-minute grocery and essential deliveries.
- **Core Analytics Objective:** Predict real-time stockout risks across urban micro-warehouses based on hyper-local weather shifts and traffic surges, while optimizing EV battery discharge and routing to minimize delivery downtime.

Why This Industry? (Quick-Commerce and Last-Mile EV Fleet Logistics)

1. Real-World Relevance and Urgency (Need of Today's Age)

- **Hyper-Fast Consumer Expectations:** Quick-commerce (10–20 minute delivery platforms like Zepto, Blinkit, Instamart) has fundamentally transformed urban retail. Success depends entirely on millisecond-level data accuracy across hyper-local micro-fulfillment centers (dark stores).
- **Sustainability and Fleet Electrification:** Cities are rapidly transitioning delivery fleets to Electric Vehicles (EVs). However, EV logistics introduce complex operations: battery drain management, charging point bottlenecks, and dynamic range calculation under heavy cargo loads.
- **High Economic Risk of Downtime:** Stockouts lead to immediate order cancellations, while dead EV batteries halt delivery operations. Managing both inventory and fleet health simultaneously requires real-time streaming, not batch processing.

2. Why Streaming (Apache Kafka) is Strictly Necessary

- **Legacy Batch Processing Fails Here:** Updating inventory or battery status every few hours causes inventory mismatches and stranded delivery drivers.
- **Low-Latency Event-Driven Architecture:** Kafka acts as the central event nervous system. It decouples high-frequency telemetry (GPS/battery every few seconds) from heavy transactional data (order checkouts) and routes them seamlessly to analytics and routing engines.

Data Sources

To support streaming into Kafka in Assignment 2, the pipeline ingests three distinct data streams:

- **IoT Telemetry Stream (Real-Time):** Electric delivery vehicle GPS coordinates, battery health state-of-charge (SoC), motor temperature, and live speed metrics.
- **Transactional and Inventory Stream (Real-Time):** Live order placement spikes, SKU-level warehouse inventory level updates, and cancellation/delay events.

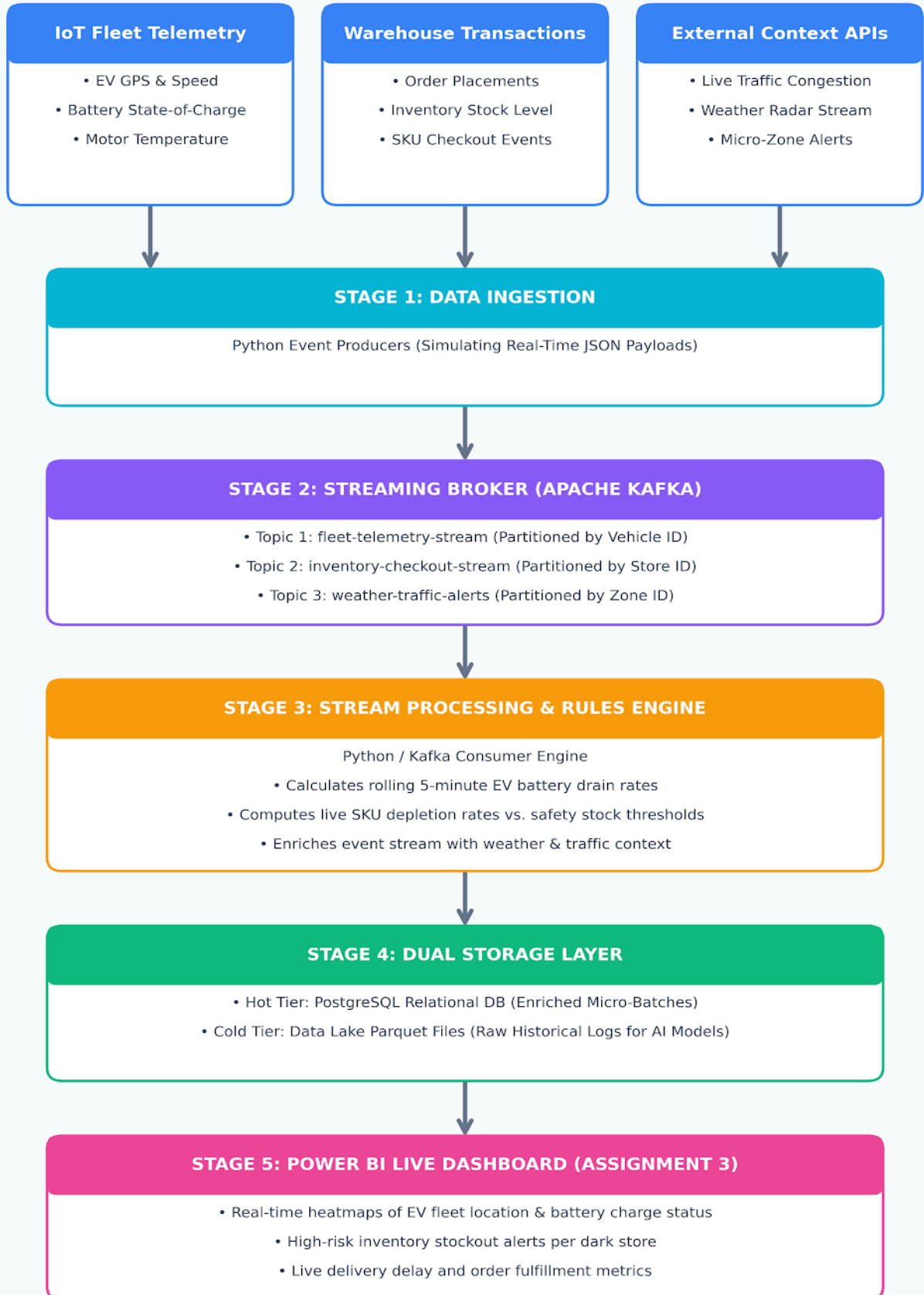
- **External APIs (Streaming/Periodic):** Hyper-local weather radar (sudden rainstorms trigger massive grocery delivery surges) and city traffic congestion feeds.

End-to-End Data Pipeline Architecture

Pipeline Stage	Technology Stack	Function / Purpose
1. Data Generation and Ingestion	Python Producers / IoT Simulators	Streams simulated EV telemetry, inventory checkouts, and weather alerts into Kafka topics.
2. Message Broker / Streaming Core	Apache Kafka	Handles high-throughput event streaming via topics like fleet-telemetry and inventory-orders .
3. Stream Processing and Analytics	Python (Kafka-Python / Pandas)	Aggregates incoming telemetry and order rates to flag critical stockouts and low battery risks in real-time.
4. Storage Layer	PostgreSQL / Time-Series DB	Maintains historical records of delivery times, route efficiencies, and inventory turnover.
5. Visualization and Dashboarding	Power BI	Renders live spatial heatmaps of delivery bottlenecks and fleet charge levels.

End-to-End Data Pipeline Architecture

Real-Time EV Logistics & Micro-Fulfillment Operations



Detailed Pipeline Components and Specifications

Pipeline Phase	Component	Technical Specification / Protocol	Data Payload / Format
Ingestion (Producer)	Fleet IoT and Store Terminals	Lightweight Python scripts pushing continuous events over HTTP/MQTT	JSON payloads containing sensor pings and stock updates
Message Queue	Apache Kafka Cluster	Multi-topic setup with key-based partitioning to maintain event ordering	Binary/JSON serialized event streams
Processing	Consumer Application	Python-based streaming script (using kafka-python or PySpark Streaming)	Structured windowed aggregations (e.g., 5-min averages)
Storage	Relational / Time-Series DB	PostgreSQL tables designed for fast query indexing on timestamps and IDs	Cleaned, normalized analytical schema
Consumption	Power BI Dashboard	DirectQuery or periodic refresh connection to the database layer	Executive visual dashboards, alert tables, and spatial heatmaps